Who wrote it? Identifying LLM's through their responses.

Maddox Nowicki
https://github.com/Maddox-Nowicki/Midterm-ProjectCMPSC448

Overview:

For this assignment, I was tasked with trying to figure out if you are able to determine which LLM wrote a message based exclusively on the text. Using the public dataset, UltraFeedback, I constructed a balanced four class problem with ChatGPT, Llama-2, Bard and Falcon and trained a CNN and a bidirectional LSTM. I was only able to run shorter experiments and could not handle the entire dataset that UltraFeedback had due to the limits of my laptop.. I analyzed 1600 responses, 2 epochs on only 1 seed. With this information the CNN was able to identify the source with 56.9% accuracy and the BiLSTM with 51.3%. Feature analysis shows that punctuation and formatting habits, response length and vocabulary diversity all carry signals. Removing formatting and capitalization from the text lowers CNN accuracy by only 1.3 points, so wording matters most. We also find dataset artifacts like chat-template tokens for Falcon, that a classifier can exploit.

Problem Definition:

As LLM-written text becomes common online, it is useful to know if we are able to easily distinguish between models. By framing this as supervised multi-class text classification: given a response (and optionally the prompt that produced it), predict which of four LLM families produced it. I investigated all four research questions.

- RQ1: Can we identify which LLM generated a response?
- RQ2: Does the user prompt help?
- RQ3: Do fingerprints generalize to task types never seen in training?
- RQ4: Which characteristics (length, vocabulary, formatting, punctuation) distinguish the LLMs?

Data Source:

I used the public UltraFeedback dataset. Each prompt in UltraFeedback has several completions written by different LLMs, and every completion is labelled with the name of the model that wrote it. Our script build_dataset.py extracts (LLM_name, LLM_input, LLM_output) triples as follows:

- Specific model names are grouped into four families by name matching (Table 1); other models are discarded.
- Responses shorter than 20 characters are removed. Prompts are truncated to 2,000 characters and responses to 4,000 characters.

• The dataset is balanced by sampling the same number of examples from every family (seed 42). We also keep a domain column, which is the UltraFeedback source tag of the prompt (used for RQ3).

LLM family (class)	Model names matched	Examples (full dataset)	Examples (quick run)
GPT	gpt-4, gpt-3.5-turbo	[N]	400
Llama-2	llama-2-*-chat	[N]	400
Bard	bard	[N]	400
Falcon	falcon-*-instruct	[N]	400
Total		[4N]	1,600

Table 1. Class definition and size. The full dataset uses up to 2,500 examples per family; the quick run is a random 400-per-family subsample.

Domain (source tag)	Short description	Examples in quick run
sharegpt	Real user conversations shared with a chatbot	512
flan	Prompts from the Flan collection of NLP tasks	402
ultrachat	Synthetic multi-turn dialogue prompts	325
evol_instruct	Complex instructions generated by evolving simpler ones	272
false_qa	Questions that contain a false premise	66
truthful_qa	Questions probing truthfulness (too few to use in RQ3)	23

Table 2. Task domains available in the data and their sizes in the quick sample.

Splits and Preprocessing:

Several LLMs answered the same prompt, so a random split would put the same prompt in both training and test data and inflate results. We therefore split by prompt (80% train / 10% validation / 10% test; in the quick run 1,278 / 162 / 160 examples). Text is tokenized with a regular expression that keeps words, punctuation marks and newlines (newlines become a special token), so that formatting information is preserved. The vocabulary is built from training data only (at most 30,000 tokens, with a minimum frequency of 2. Index 0 is padding and 1 is unknown). Responses are truncated to 256 tokens and prompts to 64 tokens; for the input and output setting the two are concatenated with a separator token.

Experiment Scope:

Training deep models on the full dataset was too slow for my laptop. We therefore report two configurations and always state which one a number comes from: quick (400 examples per family, 2 epochs, one seed) for RQ1 to RQ3, and full (all sampled data, up to 8 epochs) for the RQ4 analyses.

Models and Training Info:

CNN:

The CNN follows the standard text-classification design of Kim (2014). Tokens are mapped to 128-dimensional embeddings. Three parallel 1-D convolutions with kernel sizes 3, 4 and 5 (100 filters each, ReLU) scan the sequence; each is max-pooled over time and the three results are concatenated (300 features), passed through dropout (0.5) and a linear layer that outputs four class scores. The convolutions act like learned n-gram detectors, which suits stylistic cues such as characteristic phrases or markdown patterns.

BiLSTM:

The BiLSTM embeds tokens (128 dimensions, dropout 0.4), feeds them to a one-layer bidirectional LSTM with 128 hidden units per direction, and uses packed sequences so that padding is ignored. The hidden states are summarized by concatenating max-pooling and mean-pooling over time (512 features) followed by dropout and a linear output layer.

Training Procedures:

Both models are trained with cross-entropy loss and the Adam optimizer (learning rate 0.001, weight decay 1e-5) using mini-batches of 64 examples, and gradients are clipped at a maximum norm of 5. Inputs are truncated to 256 tokens for responses, 64 tokens for prompts and 321 tokens for the combined input + output setting. After every epoch I computed the macro-F1 score on the validation set and kept the weights from the best epoch, which are then used once on the test set. In the quick configuration, each model is trained for 2 epochs with a single random seed, so there are no standard deviations. In the full-data analyses of RQ4 training runs for up to 8 epochs and stops early if validation macro-F1 does not improve for 2 consecutive epochs. All hyperparameters are defined in config.py in the repository. Besides the two required neural models, we trained two classical baselines for the RQ4 analysis: a Random Forest on 18 hand-crafted features and a logistic regression on TF-IDF word and bigram features.

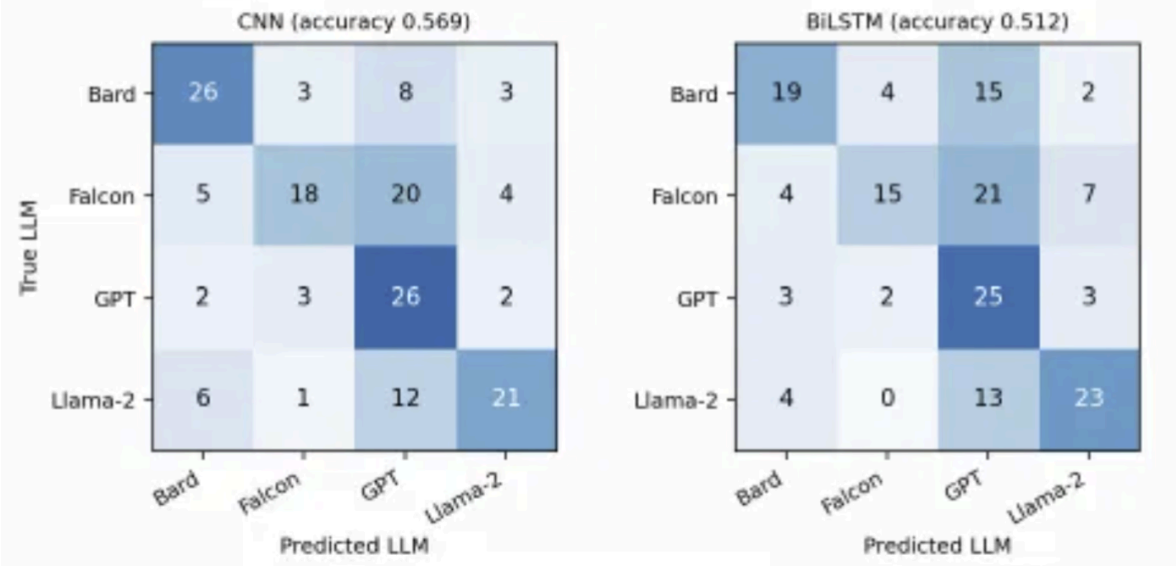
Results:

RQ1: Can we identify the LLM from its response?

Yes. Using only the response text, the CNN reaches 56.9% accuracy and the BiLSTM 51.3%), which is much better than 25% for random guessing. The CNN trained much faster (about 1 second per epoch versus 22 to 27 seconds for the BiLSTM) and was better after two epochs. The BiLSTM's training loss was still falling slowly (1.38 to 1.31), which suggests it is under-trained rather than fundamentally weaker; this is consistent with the full-data CNN results in Section 4.2, where the same architecture reaches over 80%. With more powerful hardware, the CNN could have been trained for longer and reached higher levels of accuracy.

Input to classifier	CNN accuracy	CNN macro-F1	BiLSTM accuracy	BiLSTM macro-F1
Output only (RQ1)	0.569	0.571	0.513	0.516
Input only	0.238	0.205	0.238	0.165
Input + output	0.519	0.505	0.363	0.333
Chance (4 balanced classes)	0.250	-	0.250	-

Table 4. Test performance by input type (quick run, 160 test examples, one seed).



RQ2: Does the User Prompt Help?

Can the LLM be predicted from the prompt alone?

No. Input-only accuracy is 23.8% for both models, statistically indistinguishable from chance (25%).

Does including the input improve classification?

No. Input + output is lower than output only (CNN 51.9% vs 56.9%; BiLSTM 36.3% vs 51.3%). The prompt adds uninformative tokens and longer sequences, and with very little training data the models cannot learn to ignore them; the BiLSTM suffers most.

Why is input-only at chance?

In UltraFeedback the models that answer each prompt are assigned essentially at random, so the prompt carries no information about which family answered it, and our prompt-grouped split rules out leakage through repeated prompts. This is a property of the dataset, not a general rule.

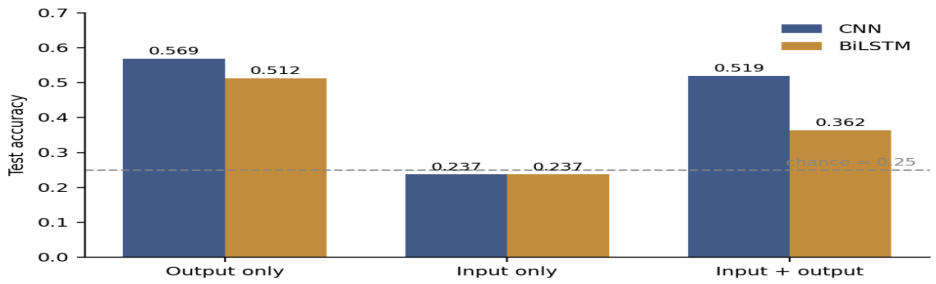


Figure 1. Test accuracy by classifier input. The dashed line is chance level.

In-depth Analysis:

RQ3: Do LLM fingerprints generalize across tasks or domains?

I held out one task domain at a time (training on all other domains, testing on the held-out one) using output-only classification and compared with the accuracy the standard-split model achieves on the same domain (quick configuration). Domains with fewer than 40 examples (truthful_qa) were skipped.

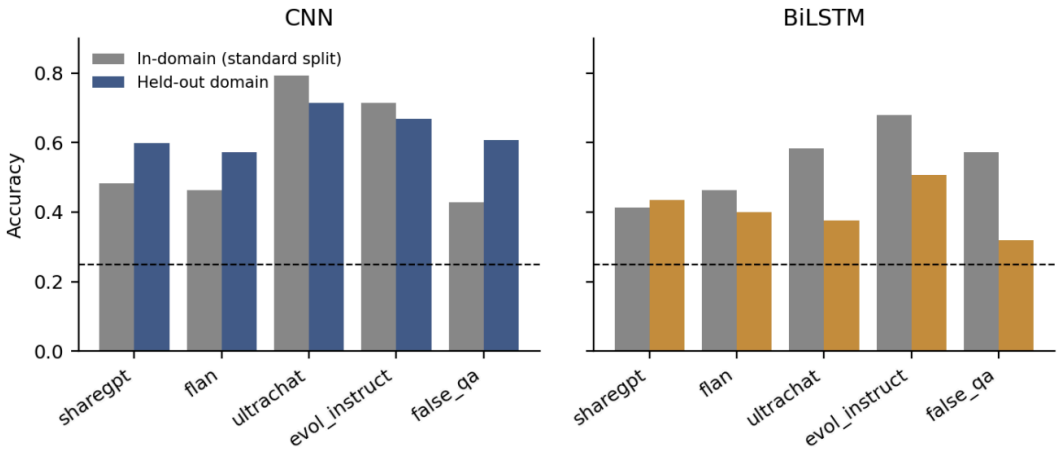


Figure 2. Accuracy on a held-out domain compared with in-domain accuracy from the standard split. The dashed line is chance (0.25).

Held-out domain	n test	CNN held-out	CNN in-domain	BiLSTM held-out	BiLSTM in-domain
sharegpt	512	0.598	0.483	0.436	0.414
flan	402	0.572	0.463	0.401	0.463
ultrachat	325	0.714	0.792	0.375	0.583
evol_instruct	272	0.669	0.714	0.507	0.679
false_qa	66	0.606	0.429	0.318	0.571
Mean		**0.632**	**0.576**	**0.407**	**0.542**

Table 5. Held-out-domain versus in-domain accuracy. In-domain values are computed on only about 7 to 60 test examples per domain, so they are very noisy.

Held-out domain	CNN Bard	CNN Falcon	CNN GPT	CNN Llama-2	BiLSTM Bard	BiLSTM Falcon	BiLSTM GPT	BiLSTM Llama-2
sharegpt	.755	.297	.785	.603	.286	.333	.252	.816
flan	.857	.641	.216	.692	.881	.117	.186	.692
ultrachat	.816	.833	.577	.588	.208	.352	.949	.044
evol_instruct	.583	.500	.868	.760	.208	.655	.691	.660
false_qa	.586	.625	.417	.889	.103	.563	.167	.778

Table 6. Per-class recall on each held-out domain.

Does performance decrease on unseen tasks?
For the CNN, not consistently: it averages 63.2% on held-out domains, and the apparent in-domain advantage disappears once we notice that the in-domain numbers rest on a handful of examples (for false_qa, roughly 7). For the BiLSTM, held-out accuracy (40.7%) is on average lower than in-domain accuracy (54.2%), but it is also the less well-trained model and its per-class recalls swing wildly (for ultrachat, Llama-2 recall is 4% while GPT recall is 95%), which looks like unstable training rather than a clean domain effect.
Are some models more consistently identifiable?
For the CNN, Bard (recall .58 to .86) and Llama-2 (.59 to .89) vary the least across domains, while GPT (.22 to .87) and Falcon (.30 to .83) vary the most. GPT recall collapses on flan (0.22), where prompts often ask for very short answers that leave little style to detect.
General fingerprint or task-specific patterns?
The CNN stays well above chance (57% to 71%) in all five unseen domains, so a substantial part of what it learns transfers across tasks, but the large swings in per-class recall show that part of the signal is task-dependent.

RQ4: What characteristics distinguish different LLMs?

For this problem I was able to run the full dataset and a prompt-grouped 80/10/10 split.

Response characteristics.
Figure 3 and Table 5 compare simple statistics of the responses. Falcon writes much shorter responses (97 words versus 212 to 270) with few paragraphs and almost no markdown. Bard uses far more markdown, with about 2.8 bold markers per response (10 to 28 times more than GPT or Llama-2). GPT uses the most numbered lists, and Llama-2 uses the most exclamation marks (3.5 times GPT's rate). The type-token ratio is partly a length effect: Falcon's high value (0.743) mostly reflects its short responses.

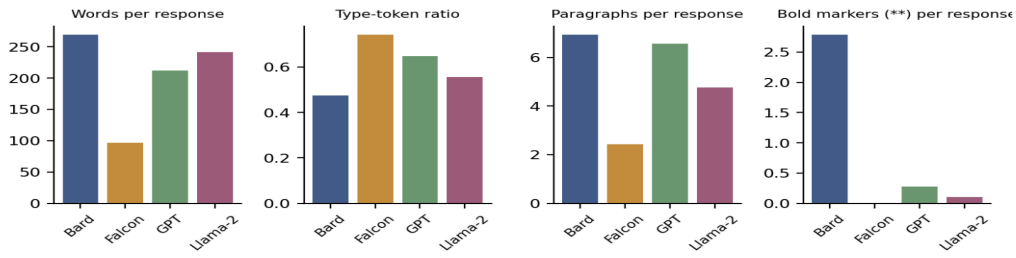


Figure 3. Mean response length, lexical diversity, paragraphs and bold markers per LLM family.

Feature (mean per response)	Bard	Falcon	GPT	Llama-2
Words	269.6	96.8	211.9	241.4
Paragraphs	6.94	2.43	6.56	4.76
Type-token ratio	0.475	0.743	0.649	0.555
Bullet points	2.81	0.34	0.79	0.84
Numbered list items	0.97	0.75	2.72	2.00
Bold markers (**)	2.79	0.008	0.28	0.10
Exclamation marks per word	0.001	0.004	0.002	0.007
Question marks per word	0.002	0.017	0.003	0.003

Table 5. Selected feature means by LLM (all features are in results/rq4_feature_means.csv).

Which signals matter?

We trained a Random Forest on 18 hand-crafted features in five groups and measured accuracy with each group removed or used alone (Figure 4). All features give 72.6%. Removing punctuation features hurts most (down 6.4 points to 66.2%), then formatting (down 3.9 points); removing length, lexical or character-mix features costs only 1.4 to 1.7 points. Formatting is the strongest single group (54.1%), but every group alone is far below the full set, so the signals are complementary: the LLMs differ in both structure and linguistic habits.

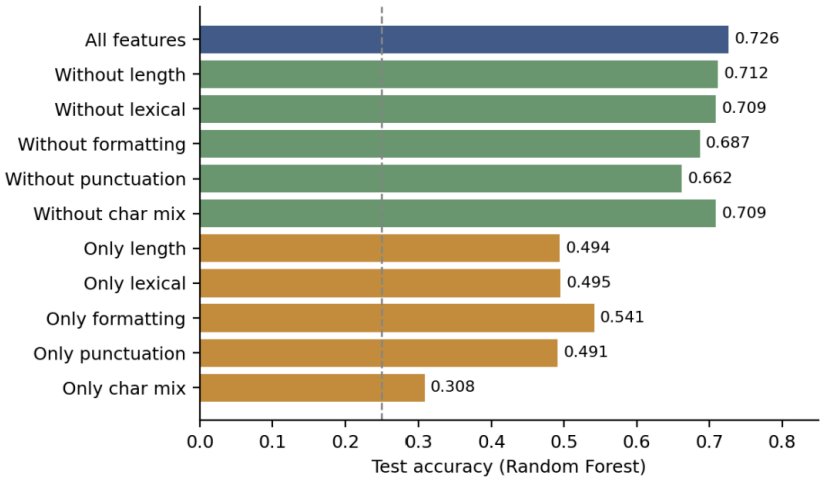


Figure 4. Random Forest accuracy with a feature group removed (green) or used alone (orange). Dashed line: chance.

Which words give the LLMs away?

Table 6 lists tokens with the highest weights in a TF-IDF logistic regression. Bard is recognized by markdown asterisks, Llama-2 by exclamations and offers to help, and Falcon by question marks and apologies. Two entries are dataset artifacts rather than writing style: Falcon's list contains chat-template markers ('assistant', 'user' and an end-of-text tag), and GPT's contains '95 %', which appears to come from confidence statements in the generation setup. A classifier can therefore score well partly by detecting how the data was collected.

LLM	Highest-weight tokens
Bard	*, * *, is a, also, :, are, is
Falcon	?, </, s > (end-of-text tag), assistant, user, sorry
GPT	: , might, like, various, 95 %
Llama-2	!, I, to help, ! I, question

Does removing signals hurt the CNN?

We removed markdown symbols, newlines and capitalization from the text and retrained the CNN. Accuracy fell only from 81.4% to 80.1% (Figure 5), so the CNN relies mostly on word choice and phrasing rather than on formatting. A plain TF-IDF logistic regression (83.0%) is slightly better than the CNN, which shows that bag-of-words features already capture most of the fingerprint.

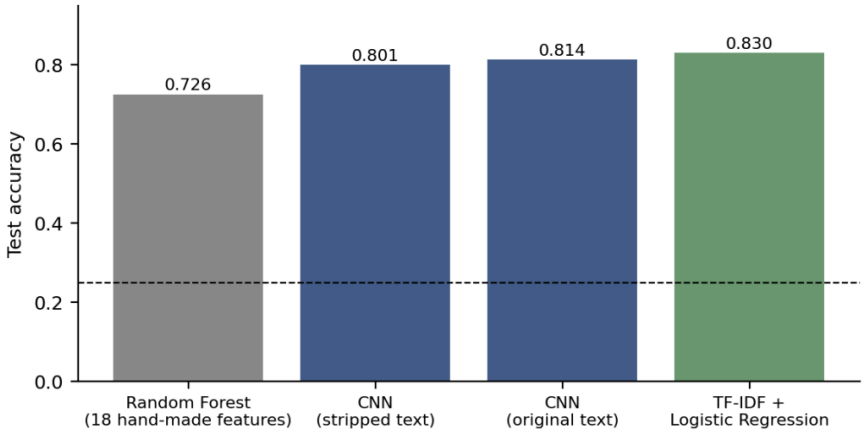


Figure 5. Test accuracy of the classifiers on the full dataset (output only). Dashed line: chance.

What I learned:

Split by prompt.
Several LLMs answer the same prompt, so grouping the split by prompt is needed to avoid leakage. I would not have noticed this without thinking about how the data was created.
Understand what the dataset can answer.
Input-only accuracy was at chance because UltraFeedback assigns models to prompts at random. A different collection process would have given a different answer to RQ2, so the result tells us about the dataset as much as about LLMs.
Hardware is a significant factor.
On my laptop the BiLSTM was about 20 times slower per
epoch than the CNN. I had to use a small quick configuration for RQ1 to RQ3, which made the BiLSTM look weaker and made results noisy; with stronger hardware I could run several seeds and report standard deviations.
Beware of artifacts.
Chat-template tokens and confidence statements show that high accuracy does not automatically mean a model learned 'writing style'. Inspecting discriminative tokens was essential to catch this.
Small test sets are noisy.
With 160 test examples, differences of a few points are within the margin of error, and per-domain in-domain baselines were based on as few as 7 examples.